

CS 218 — Programmable & Interoperable Blockchain

Project Marks Allocation — Detailed Breakdown

Total: 40 Marks per Project (25 Submission + 15 Presentation) · Teams of up to 6 · Presentation: 25–26 April 2026 · Submission Deadline: 10 May 2026, 11:59 PM

How This Document Is Organised

The total 40 marks per project are split as follows: 25 marks for the Project Submission (deadline: 10 May 2026, 11:59 PM) and 15 marks for the Project Presentation (25–26 April 2026). The submission rubric below is marked out of 25. The presentation is graded separately out of 15 — see the Presentation Rubric section below. The submission rubric is identical for all 10 projects. This document translates that rubric into project-specific guidance — for every project, you will find exactly which functions earn which marks, what access control and edge cases are expected, which tests must pass, and what the frontend must demonstrate.

Presentation Rubric — 15 Marks (25–26 April 2026)

Criterion	Max	What Earns the Mark	Marks
Live Demo (Working DApp Demonstration)	6	Smart contract deploys and all core functions execute correctly on-screen (3). MetaMask wallet connects and at least one state-changing transaction is demonstrated live via the DApp frontend (3).	6
Technical Explanation (Design & Architecture)	5	Team clearly explains the contract architecture, key design decisions (on-chain vs off-chain, access control), and security measures taken (2). Gas optimisation approach is presented with before/after numbers (2). On-chain vs off-chain data choices are justified (1).	5
Q&A / Understanding	4	Each team member can answer questions on their own code (2). Team demonstrates understanding of blockchain concepts underpinning their project (reentrancy, gas, IPFS, GDPR) (2).	4
PRESENTATION TOTAL			15

Project Submission Rubric — 25 Marks (Deadline: 10 May 2026, 11:59 PM)

Category	Max	Sub-Criteria	Marks
Smart Contract Correctness	9	Core logic correct (all required functions implemented and behave correctly)	5
		Access control (only authorised addresses can call privileged functions)	2

Category	Max	Sub-Criteria	Marks
		Edge cases handled (zero values, expired deadlines, empty arrays, etc.)	2
Security	4	Reentrancy guard on all ETH-sending functions	2
		Input validation — all require() statements; no over/underflow	1
		Visibility correct (public/private/internal/external assigned well)	1
OpenZeppelin Usage	1	OZ contracts used where applicable (Ownable, ReentrancyGuard, token standards)	1
Gas Optimisation	3	Gas report submitted (from hardhat-gas-reporter or forge --gas-report)	1
		One concrete optimisation shown with before/after numbers and explanation	2
Testing	4	Happy-path tests for every core function	2
		Revert/failure tests (unauthorised or invalid calls revert correctly)	1
		Coverage report showing 70% or greater line coverage	1
DApp Frontend	3	Wallet connection (MetaMask, contract ABI loaded, balance displayed)	1
		At least one state-changing function callable from UI with status feedback	2
Documentation & Code	1	README complete (setup steps, team members, roll numbers) + NatSpec comments (@notice, @param) on all public functions	1
SUBMISSION TOTAL			25

Deductions (applied on top of the 25 submission marks above):

- **-2 marks** GitHub repository is private at submission time.
- **-3 marks** Contract does not compile.
- **-1 mark each** Missing team member name in README.
- **-2 marks** No tests are submitted.

The Cardinal Rule: On-Chain vs Off-Chain Data

Blockchain storage is expensive, permanent, and public. Every byte stored on-chain costs real gas and is visible to every node on the network forever. This has direct consequences for cost, privacy, and legal compliance.

Store ON-CHAIN when data...	Keep OFF-CHAIN when data...
Is required by smart contract logic (e.g. bid amount, deadline)	Is a human-readable description, title, or image
Must be verified or enforced without trusting any server	Can be verified by storing only its keccak256 hash on-chain
Is a financial balance or token ownership record	Contains personal details (name, email, ID number, address)
Is an access control record (who is a verifier, who is an owner)	Is subject to GDPR right-to-erasure — immutability prevents deletion
Is a cryptographic commitment (a hash, not the secret itself)	Is large in size — images, PDFs, JSON metadata, documents

The IPFS pattern (mandatory for content data): Store your file or JSON on IPFS, get back a CID (a 46-character content hash). Store **ONLY** that CID on-chain. Anyone can verify the content by fetching from IPFS and hashing it — the on-chain CID is the proof of authenticity.

Gas cost at a glance: SSTORE (new slot) = 20,000 gas. Each non-zero calldata byte = 16 gas. A 500-character description stored as a string on-chain is approximately 40,000 gas. The same description on IPFS with only a 32-byte CID on-chain is approximately 512 gas. That is a 78x reduction.

GDPR and immutability: Blockchain data is permanent. GDPR Article 17 gives users the right to have their personal data erased. These two facts are irreconcilable if personal data is stored on-chain. The solution is always: store personal data off-chain (encrypted, deletable), store only a non-reversible hash on-chain.

Project 1 — Decentralised Book Rental

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
rentItem() payable	2	Validates payment = deposit + first-day fee; status → Rented; ETH held in contract
resolveDispute()	1	Arbitrator-only; routes ETH to correct party; emits event
confirmReturn()	1	Refunds deposit to renter; status → Closed; enforces 48-hour window
listItem() + returnItem() + raiseDispute() + getItem()	1	Supporting functions: struct stored correctly, status transitions enforced, events emitted
Access Control	2	Only owner can list items. Only arbitrator can resolve disputes. Only renter can raiseDispute after return.
Edge Cases	2	Zero-value rent attempt; renting an already-rented item; 48-hour auto-refund if owner does not confirmReturn.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: rentItem() and resolveDispute() (both send or receive ETH).
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	Ownable (listing); ReentrancyGuard (rentItem, resolveDispute).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Non-owner cannot list item • Renting already-rented item reverts • Happy path: <code>list</code> → <code>rent</code> → <code>confirmReturn</code> → deposit refunded • Dispute path: <code>raiseDispute</code> → <code>resolveDispute</code> in owner's favour → deposit to owner • 48-hour timeout: advance time, auto-refund, verify renter balance
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>rentItem()</code> — user selects an item and pays deposit + rental fee via MetaMask.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING Owner and renter wallet addresses are publicly visible on-chain. Do NOT store real names, phone numbers, or physical addresses. Use the IPFS CID pattern: store keccak256(CID) on-chain for verifiability.

GDPR / Privacy: *Physical item descriptions may indirectly identify users. Keep all personal details off-chain.*

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Item ID, owner address, renter address, pricePerDay (wei), depositAmount (wei)	Item name, description, photos — store on IPFS; put only the CID on-chain
Status enum (Available / Rented / AwaitingConfirm / InDispute / Closed)	Condition reports and damage photos submitted at return — store on IPFS
rentedAt timestamp, disputeRaisedAt timestamp	User identity / KYC documents — never on-chain under any circumstance
ETH balances locked inside the contract	Chat or negotiation logs between owner and renter

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 2 — Decentralised Freelance Platform

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
hireFreelancer() payable	2	Exact price escrowed; emits JobCreated; status → Hired
confirmCompletion()	1	Client-only; releases ETH to freelancer; emits JobCompleted
rateFreelancer() + getReputation()	1	1-5 score stored; prevents double-rating; returns (avgScore x 100, totalJobs) without decimals
offerService() + cancelJob()	1	Service struct stored; either party can cancel before deadline; client refunded
Access Control	2	Only client can confirmCompletion. Freelancer cannot self-release funds. Only client can rate after completion.
Edge Cases	2	7-day auto-cancel if no action; rating same job twice reverts; hiring a non-Listed service reverts.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: hireFreelancer() and confirmCompletion() (ETH movements).
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ReentrancyGuard (hireFreelancer, confirmCompletion).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	- Only client can confirm completion - Freelancer cannot self-confirm or self-release funds - Rating the same job twice reverts 7-day auto-cancel: advance time, cancel, verify client refund - Reputation average: two jobs rated 4 and 2 → average = 3
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>hireFreelancer()</code> — client selects a service and sends the exact price via MetaMask.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING Rating scores and wallet addresses are permanently public on-chain. Do NOT store freelancer personal details, banking info, or email on-chain. Reputation is tied to the wallet address, not a real name.

GDPR / Privacy: *Linking a wallet to a real person's portfolio violates GDPR if that portfolio contains personal data. Keep profile data off-chain and reference only by IPFS CID.*

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Service ID, freelancer address, client address, priceWei	Service title, full description, portfolio samples — store on IPFS; put CID on-chain
Job status enum (Active / Done / Cancelled)	Deliverable files submitted by freelancer — store on IPFS; hash the CID on-chain
Rating score (uint8 1-5), job completion timestamp	Communication between client and freelancer
Escrowed ETH held inside the contract	Freelancer profile photo, resume, credentials

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 3 — DAO Governance Contract

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
createProposal()	1	Snapshots voter balances at creation; stores target + calldata; emits event
vote()	1	One vote per address; weight = snapshot balance; prevents double-vote
executeProposal()	2	Checks deadline passed, quorum ($\geq 30\%$) met, majority yes; calls target contract
cancelProposal() + MintableToken (ERC-20)	1	Creator-only cancel before voting starts; governance token deployed and distributed
Access Control	2	Only token holders can vote. Only creator can cancel. Anyone can execute after deadline if conditions met.
Edge Cases	2	Non-token-holder cannot vote; proposal below quorum cannot execute; defeated proposals cannot execute; voting after deadline reverts.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: executeProposal() — makes an external call to target contract.
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ERC20 (governance token); AccessControl (ADMIN_ROLE for quorum update).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Non-token-holder cannot vote • Double-voting on same proposal reverts • Proposal below quorum cannot execute even with all-yes votes • <code>executeProposal</code> reverts before deadline, succeeds after • Successful vote triggers real state change in Treasury contract
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>vote()</code> — token holder selects a proposal and casts yes/no vote via MetaMask.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING All votes and voter addresses are publicly visible. Do NOT store voter opinions, private deliberations, or any off-chain identity link. The description field should be an IPFS CID, not a raw string — raw strings cost ~68 gas per byte and bloat the chain.

GDPR / Privacy: *Voting records tied to wallet addresses are pseudonymous but potentially identifiable. Never associate wallet addresses with real-world voter identities on-chain.*

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Proposal ID, target contract address, encoded calldata (bytes)	Full proposal text, rationale, forum discussion — store on IPFS; put CID on-chain as description field
Voting deadline timestamp, yes/no vote counts	Supporting documents, audit reports referenced in the proposal
Voter address to has-voted mapping (bool)	Off-chain signalling votes (Snapshot.org style) before on-chain binding vote
Snapshot of token balances at proposal creation time	
Quorum threshold percentage	

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 4 — Lottery System with Commit-Reveal Randomness

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
buyTicket() payable	1	Adds msg.sender to participants; reverts after sale closed
commitHash(bytes32)	1	Owner submits keccak256(secret); phase → Committed
revealAndDraw(bytes32)	2	Verifies hash match; computes winner index via deterministic formula; phase → Drawn
claimPrize() + closeSale()	1	Winner-only prize withdrawal; owner-only phase transition to SaleClosed
Access Control	2	Only owner can closeSale, commitHash, revealAndDraw. Only winner can claimPrize.
Edge Cases	2	Wrong secret reveals revert (hash mismatch); second revealAndDraw after winner set reverts; non-winner claimPrize reverts; ticket purchase after close reverts.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: claimPrize() (sends the full prize pool as ETH).
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ReentrancyGuard (claimPrize); Ownable (phase transitions).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Ticket purchases revert after sale is closed • Revealing wrong secret (hash mismatch) reverts • Only winner can claim the prize • Prize pool exactly equals sum of all ticket payments • Second call to <code>revealAndDraw</code> reverts; non-winner <code>claimPrize</code> reverts
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>buyTicket()</code> — user purchases a lottery ticket and pays the ticket price via MetaMask.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING The commit-reveal scheme exists precisely to keep the winning secret off-chain until the draw is locked. If the secret were stored on-chain before reveal, any miner could read it and front-run the outcome. Also: participant addresses are public — do NOT store participant names or emails alongside addresses.

GDPR / Privacy: Participant wallet addresses are permanently on-chain. Never link them to personal identifiers. The lottery is pseudonymous by design.

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Ticket IDs and participant wallet addresses	The raw secret value — never stored on-chain until the reveal step, and even then only transiently in the transaction calldata
Phase enum (Open / SaleClosed / Committed / Drawn)	Lottery promotional content, ticket artwork — store on IPFS
Committed hash (bytes32 keccak256 of the secret)	Any personal details used for KYC (if applicable)
Winner address, prize pool amount (wei)	

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 5 — Decentralised Auction System

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
createAuction()	1	Stores item struct with deadline; returns auctionId
placeBid() payable	2	Must exceed highest bid; previous high bid added to pendingReturns mapping
endAuction()	1	After deadline; transfers highest bid to seller; emits AuctionEnded
withdrawBid()	1	Withdrawal pattern; losing bidder pulls their own ETH
Access Control	2	endAuction callable by anyone after deadline. No onlyOwner needed — auctions are permissionless.
Edge Cases	2	Bid below current highest reverts; endAuction before deadline reverts; second call to endAuction reverts; bids after endAuction reverts.
Sub-total		9 marks

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: withdrawBid() — classic reentrancy target; must be guarded.
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ReentrancyGuard (withdrawBid).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report. Paste the output table into report.pdf.

Marks	Task	How to Do It
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use uint256 instead of uint8 in loops, mark a function pure/view). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	- Bids below current highest revert - endAuction reverts before deadline - Losing bidders withdraw their exact bid amount - Reentrancy test: malicious contract re-enters withdrawBid; guard prevents drain - After endAuction, no further bids accepted; seller receives highestBid
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: placeBid() — user enters a bid amount and sends ETH to the auction contract via MetaMask.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (@notice, @param, @return) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING All bid amounts and bidder addresses are publicly visible on-chain in real time — this is a known weakness of open English auctions (competitors can see your maximum bid). Do NOT store seller personal information or item serial numbers that could be used for fraud. Item photos must go on IPFS, not calldata.

GDPR / Privacy: Bidder wallet addresses are permanently public. Never store bidder names, addresses, or payment details on-chain.

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Auction ID, seller address, highest bidder address, highest bid amount (wei)	Item name, full description, photos — store on IPFS; reference by CID in the contract
Auction deadline timestamp, ended bool	Item condition reports, provenance documents
pendingReturns mapping (address to wei owed back to losing bidders)	Seller's contact details or shipping information — never on-chain

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 6 — Decentralised Identity Verification

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
registerIdentity(bytes32)	1	Stores keccak256 hash on-chain ONLY; status → Pending; emits event
verifyIdentity(address)	1	Verifier-only; status → Verified; records verifiedBy address
revokeIdentity(address)	1	Verifier-only; status → Revoked; emits IdentityRevoked
KYCGatedAuction contract	2	Calls isVerified(msg.sender) inside placeBid; reverts with 'KYC required' if false — demonstrates composability
Access Control	2	Only admin (owner) can add/remove verifiers. Only verifiers can verify or revoke. isVerified is public/external for composability.
Edge Cases	2	Unregistered user not verified; revoking unregistered user reverts; non-verifier cannot approve; unverified address bid in KYCGatedAuction reverts.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: No direct ETH transfers in identity contract; guard on KYCGatedAuction's placeBid.
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	Ownable or AccessControl (ADMIN_ROLE, VERIFIER_ROLE).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Non-verifier cannot approve identities • Unregistered user is not verified • Revoking verified identity changes status to Revoked • Composability: unverified address bid in <code>KYCGatedAuction</code> reverts • After verification, same address can bid successfully
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>registerIdentity()</code> — user submits a hash of their document via MetaMask to register on-chain.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING THIS IS THE MOST SENSITIVE PROJECT. Storing any identity document or personal data on a public blockchain is a critical GDPR violation. The correct pattern: hash the document off-chain using keccak256, store ONLY the hash on-chain. Blockchain immutability means you CANNOT delete data once on-chain — making it impossible to comply with GDPR's Right to Erasure if personal data is stored on-chain.

GDPR / Privacy: *GDPR Article 17 (Right to Erasure) is impossible to satisfy on a public blockchain. The hash-only pattern is the industry-standard solution: the hash proves the document existed and was verified, but reveals nothing about the document itself. Explain this in your report.*

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
keccak256 hash of the identity document — NOT the document itself	The actual identity document (passport, Aadhaar, driving licence) — NEVER on-chain
Verification status enum (NotRegistered / Pending / Verified / Revoked)	Document images, biometric data — store encrypted off-chain (e.g. IPFS with encryption)
Verifier address who approved, timestamp of verification	Personal details: full name, date of birth, address — all off-chain
Mapping of authorised verifier addresses	Verification audit trail and correspondence with verifiers

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 7 — NFT Event Ticketing (ERC-721)

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
createEvent()	1	Stores event struct including maxTickets and royaltyBps
buyTicket() payable	1	Mints ERC-721 token; transfers ETH to organiser; reverts beyond maxTickets
buyResaleTicket() payable	2	Royalty deducted from sale price; seller receives remainder; organiser receives royalty
listForResale() + cancelResaleListing() + royaltyInfo()	1	Holder listing; cancellation; EIP-2981 royaltyInfo returns (receiver, royaltyAmount)
Access Control	2	Only ticket holder can listForResale and cancelResaleListing. Only organiser deploys and creates events.
Edge Cases	2	Buying beyond maxTickets reverts; purchasing cancelled listing reverts; royalty must actually be deducted in buyResaleTicket (not just royaltyInfo).
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: buyTicket() and buyResaleTicket() (both transfer ETH).
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ERC721URIStorage; IERC2981; ReentrancyGuard on buy functions.

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Buying beyond <code>maxTickets</code> reverts • Non-owner cannot list ticket for resale • Royalty split: 1 ETH resale at 10% → seller 0.9 ETH, organiser 0.1 ETH • ERC-721 <code>ownerOf()</code> updates correctly after each transfer • Cancelled listing cannot be purchased
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>buyTicket()</code> — user selects an event and purchases an NFT ticket by paying via MetaMask; display <code>tokenId</code> .

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING Storing metadata JSON directly on-chain costs thousands of gas per ticket. The correct pattern is `tokenURI = 'ipfs://<CID>'` where the CID points to the JSON off-chain. NFT ownership (who holds the token) is public — do NOT encode attendee names or seat numbers inside on-chain token data.

GDPR / Privacy: *NFT token ownership is permanently public. If ticket holder identity must be verified at the venue, do that off-chain. Never store the attendee's real name or contact details in the NFT metadata on-chain.*

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Token ID, event ID, organiser address, owner address	Token metadata JSON (name, description, image URL) — store on IPFS; tokenURI returns the IPFS CID
Royalty basis points (uint96), resale listing price (wei)	Ticket artwork, event poster images — store on IPFS
maxTickets, ticketsSold counter, priceWei per ticket	Attendee personal details (name, email for confirmation) — off-chain ticketing backend only
tokenURI pointing to IPFS CID (a string pointer, not the metadata itself)	Event venue details, schedule, terms — off-chain

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 8 — Decentralised Crowdfunding

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
createCampaign()	1	Goal > 0; deadline in future; returns campaignId
contribute() payable	1	Active campaign only; adds to contributions mapping; emits Contributed
withdrawFunds()	2	Creator-only; after deadline; goal must be met; emits FundsWithdrawn
refund()	1	Contributor calls if deadline passed and goal not met; exact amount returned
Access Control	2	Only creator can withdrawFunds. Any contributor can refund on failed campaign. No contributions after deadline.
Edge Cases	2	Creator cannot withdraw before deadline (even if goal met); contributing after deadline reverts; zero-contribution refund attempt reverts; double-withdraw reverts.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: withdrawFunds() and refund() (both send ETH).
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ReentrancyGuard (withdrawFunds, refund).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test</code> (with <code>hardhat-gas-reporter</code>) or <code>forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Successful campaign: goal reached, creator withdraws, contributors cannot refund • Failed campaign: deadline passes, each contributor gets exact contribution back • Creator cannot withdraw before deadline even if goal is met • Contributing after deadline reverts • Zero-contribution refund attempt reverts
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>contribute()</code> — user selects a campaign and sends ETH contribution via MetaMask; display raised vs goal.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING Contribution amounts and backer wallet addresses are publicly visible — competitors and the creator can see exactly who contributed and how much. Do NOT store backer personal details (name, email, shipping address) on-chain. Campaign description should be an IPFS CID: storing a 500-word description as a string on-chain costs ~34,000 gas per campaign.

GDPR / Privacy: Contribution records linked to wallet addresses are permanently public. Never associate backer wallet addresses with personal identifiers on-chain.

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Campaign ID, creator address, goalWei, deadlineTimestamp, raisedWei	Campaign title, full description, images, pitch video — store on IPFS; put CID on-chain
Status enum (Active / Successful / Failed / FundsWithdrawn)	Backer reward tiers and fulfilment details — off-chain backend
contributions mapping (campaignId to address to uint256 amount)	Creator identity and verification documents — off-chain KYC
Milestone structs (if implementing the advanced extension)	Campaign updates and backer communications

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 9 — ERC-20 Token with Staking Rewards

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
CS218Token (ERC-20)	1	mint() restricted to Staking contract and owner; 18 decimals; standard ERC-20 interface
stake(uint amount)	1	Transfers tokens from user to contract; records (amount, startTimestamp)
claimRewards()	2	Computes accrued rewards using formula; mints to user; resets accrual timer; accounts for 18 decimals
unstake() + getPendingRewards() + getStakedBalance()	1	Returns principal; view functions return correct values; reward formula consistent
Access Control	2	Only Staking contract and owner can mint tokens. Only owner can setRewardRate.
Edge Cases	2	Staking 0 tokens reverts; immediate unstake earns 0 rewards; second immediate claimRewards returns 0; non-owner setRewardRate reverts.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: unstake() and claimRewards() (move tokens or trigger minting).
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ERC20 (token); Ownable on both contracts; ReentrancyGuard (unstake, claimRewards).

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Staking transfers tokens from user to contract (check both balances) • Rewards after 1 day are half the rewards after 2 days (linear accrual) • Staking 2x tokens earns 2x rewards in same period • <code>claimRewards</code> resets accrual timer; second immediate claim returns 0 • Unstaking returns exactly staked amount with no reward included
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>stake()</code> — user approves tokens and stakes an amount via MetaMask; display staked balance and pending rewards.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING Staked balances and reward accrual are fully public. Any wallet can read any other wallet's staked amount and pending rewards. Do NOT store user personal details, tax information, or jurisdiction data on-chain. Reward rate changes are public — document the governance implications in your report.

GDPR / Privacy: *Token holdings are pseudonymous but can be de-anonymised via on-chain analytics. Never link wallet addresses to real identities on-chain.*

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Staker address, staked amount (uint256 with 18 decimals), lastClaimTimestamp	Token project whitepaper, branding, website content — off-chain
Reward rate (tokensPerDayPer1000Staked), set by owner	User KYC for regulatory compliance (if applicable) — off-chain only
Token balances managed by the ERC-20 contract	Historical staking analytics and dashboards — off-chain indexer (e.g. The Graph)
Total staked supply (for pool-wide calculations)	

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks

Project 10 — Multi-Signature Wallet

A. Smart Contract Correctness — 9 Marks

Function	Marks	What Earns the Mark
submitTransaction()	1	Owner-only; stores Transaction struct; returns txId
approveTransaction()	1	Owner-only; increments approvalCount; prevents double-approve by same owner
executeTransaction()	2	$\geq M$ approvals required; low-level call with calldata; marks executed BEFORE the call (CEI pattern)
constructor() + revokeApproval() + receive()	1	Validates owners (no duplicates, no zero-address); revoke decrements count; receive() accepts ETH
Access Control	2	Only owners can submit, approve, revoke, and execute. Non-owners revert on all operations.
Edge Cases	2	Fewer than M approvals: execution reverts; already-executed tx cannot execute again; duplicate owners in constructor revert; double-approve reverts.
Sub-total	9 marks	

B. Security — 4 Marks

Security Criterion	Marks	Project-Specific Application
Reentrancy Guard	2	Apply ReentrancyGuard to: executeTransaction() — makes an arbitrary external call.
Input Validation	1	All require() statements for invalid inputs, zero values, and unauthorised callers.
Visibility Modifiers	1	public/private/internal/external assigned correctly on all functions.

C. OpenZeppelin Usage — 1 Mark

Marks	Required OZ Contracts for This Project
1	ReentrancyGuard (executeTransaction); written from scratch otherwise.

D. Gas Optimisation — 3 Marks

Marks	Task	How to Do It
1	Gas Report	Run: <code>npx hardhat test (with hardhat-gas-reporter) or forge test --gas-report</code> . Paste the output table into <code>report.pdf</code> .
2	Before/After Optimisation	Identify the most expensive function. Make one targeted change (e.g., cache a storage read, use <code>uint256</code> instead of <code>uint8</code> in loops, mark a function <code>pure/view</code>). Record before and after gas numbers and explain the reason in the report.

E. Testing — 4 Marks

Marks	Required Tests for This Project
2	• Non-owner cannot submit or approve transactions • Transaction with fewer than M approvals cannot execute • Full happy path: <code>submit</code> → M owners approve → execute → verify target state change • Revoking approval decrements count and blocks execution until re-approved • Duplicate owners in constructor reverts; ETH receipt and send-out works
1	Revert/failure test for each function above (wrong caller, invalid input, wrong state).
1	Coverage report showing $\geq 70\%$ line coverage (forge coverage or hardhat-coverage).

F. DApp Frontend — 3 Marks

Marks	Frontend Requirement
1	MetaMask wallet connection + contract ABI loaded + current contract state displayed (e.g. balance, status).
2	State-changing function callable from UI with status feedback: <code>approveTransaction()</code> — each owner connects MetaMask and approves a pending transaction; display approval count vs required.

G. Documentation & Code — 1 Mark

Marks	Requirement
1	README.md with setup steps (npm install, compile, test, deploy commands), team member names and roll numbers. NatSpec comments (<code>@notice</code> , <code>@param</code> , <code>@return</code>) on all public functions.

H. On-Chain vs Off-Chain Data — Cost & Privacy

WARNING ALL calldata submitted via submitTransaction() is permanently public on-chain — anyone can decode it using the contract ABI. If a transaction encodes sensitive parameters (e.g. a salary payment amount, a private key rotation), those parameters are visible to the entire world. Design your transaction payloads accordingly. Owner wallet addresses are also permanently public.

GDPR / Privacy: Multi-sig owner addresses are public. Gnosis Safe (the production multi-sig) addresses this by keeping owner identity off-chain. Never store owner personal details, legal names, or jurisdiction information on-chain.

STORE ON-CHAIN (required for contract logic)	KEEP OFF-CHAIN (IPFS / server / encrypted store)
Owner addresses array, requiredApprovals threshold (M of N)	Transaction rationale, meeting minutes, governance discussion — off-chain (e.g. Notion, IPFS)
Transaction struct: to address, value (wei), calldata (bytes), approvalCount, executed bool	Owner identity documents for multi-sig setup — off-chain KYC
Approval mapping (txId to owner address to bool)	Sensitive calldata payloads — encoded function call parameters become permanently public on-chain
ETH balance held in the contract	

Gas cost context: Every byte of calldata costs 16 gas (non-zero) or 4 gas (zero). Storing a single 32-byte slot costs 20,000 gas (SSTORE). A typical 500-character string stored on-chain adds ~40,000 gas per transaction. Use IPFS CIDs (32-byte hashes) on-chain and keep full content off-chain.

Correctness	Security	OpenZeppelin	Gas	Testing	Frontend	Docs
9 marks	4 marks	1 marks	3 marks	4 marks	3 marks	1 marks